The 15-Factor methodology promotes the *API first* pattern. It encourages you to establish the service interface first and work on the implementation later. The API represents a public contract between your application and its consumers, and it's in your best interests to define it first thing.

Suppose you agree on a contract and define the API first. In that case, other teams can start working on their solutions and develop against your API to implement their integration with your application. If you don't develop the API first, there will be a bottleneck, and other teams will have to wait until you're done with your application. Discussing the API up front also enables productive discussions with stakeholders, which can lead to you clarifying the scope of the application and even defining user stories to implement.

In the cloud, any application can be a backing service for another one. Adopting an API first mentality will help you evolve your applications and adapt them to future requirements.

This section will guide you by defining a contract for Catalog Service as a REST API, the most used service interface model for cloud native applications. You'll use Spring MVC to implement the REST API, validate it, and test it. I'll also outline some considerations for evolving an API for future requirements, a common issue in highly distributed systems like cloud native applications.

3.3.1 REST API first, business logic later

Designing the API first assumes you have already defined the requirements, so let's start with those. Catalog Service will be responsible for supporting the following use cases:

- View the list of books in the catalog.